



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

**Constituent of Symbiosis International (Deemed University),
Pune**

Subject: Computer Vision Laboratory

Experiment - 11

Aim: To build a custom CNN architecture for image classification.

Software Tools Required: Google Colab

Theory:

1. Introduction to CNNs

A Convolutional Neural Network (CNN) is a type of deep learning model designed to automatically learn patterns and features from images. CNNs are highly effective for image classification because they can detect low-level features (like edges and textures) in early layers and high-level features (like shapes or objects) in deeper layers.

2. Architecture of a Custom CNN

A typical CNN for image classification consists of the following layers:

2.1 Input Layer

- Takes the image data as input.
- Image size is usually standardized (e.g., 32×32 pixels with 3 color channels for RGB images).
- Pixel values are normalized for faster and stable learning.

2.2 Convolutional Layer

- Applies small filters to the image to extract important features such as edges, corners, and textures.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

- Each filter produces a feature map representing the presence of a feature in the image.

2.3 Activation Layer

- Introduces non-linearity so the network can learn complex patterns.

Constituent of Symbiosis International (Deemed University), Pune

- Common choice is ReLU, which keeps positive values and sets negative values to zero.

2.4 Pooling Layer

- Reduces the size of feature maps to lower computation and control overfitting.
- Common types:
 - Max Pooling: Takes the largest value from a small region.
 - Average Pooling: Takes the average value from a small region.

2.5 Fully Connected Layer

- After convolution and pooling, the feature maps are flattened into a one-dimensional vector.
- Dense layers map these features to the final output classes.

2.6 Output Layer

- Produces a probability for each class.
- Uses Softmax activation for multi-class classification.

3. Key Concepts in Designing Custom CNNs

- Depth: Number of convolutional layers. Deeper networks can learn more complex features.
- Width: Number of filters in each layer. More filters can capture more diverse patterns.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

- Regularization: Helps prevent overfitting. Techniques include:
 - Dropout: Randomly disables neurons during training.
 - Batch Normalization: Stabilizes learning and speeds up training.
 - Data Augmentation: Creates new training images by rotating, flipping, or cropping existing images.

Constituent of Symbiosis International (Deemed University), Pune

- Optimization: Choosing the right optimizer and learning rate is crucial for effective training. Common optimizers are Adam, SGD, and RMSProp.

4. Advantages of a Custom CNN

- Tailored to your specific dataset.
- Flexible architecture allows balancing accuracy and computational efficiency.
- Can include advanced features like additional layers, residual connections, or attention mechanisms.

5. Steps to Build a Custom CNN

1. Preprocess Data: Resize, normalize, and augment images.
2. Define CNN Layers: Decide the number of convolution, pooling, and dense layers.
3. Compile Model: Choose an optimizer, loss function, and performance metrics.
4. Train Model: Fit the model to training data and validate with test data.
5. Evaluate Performance: Use accuracy, confusion matrix, and other metrics.
6. Fine-Tune Model: Adjust the architecture or parameters to improve performance.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

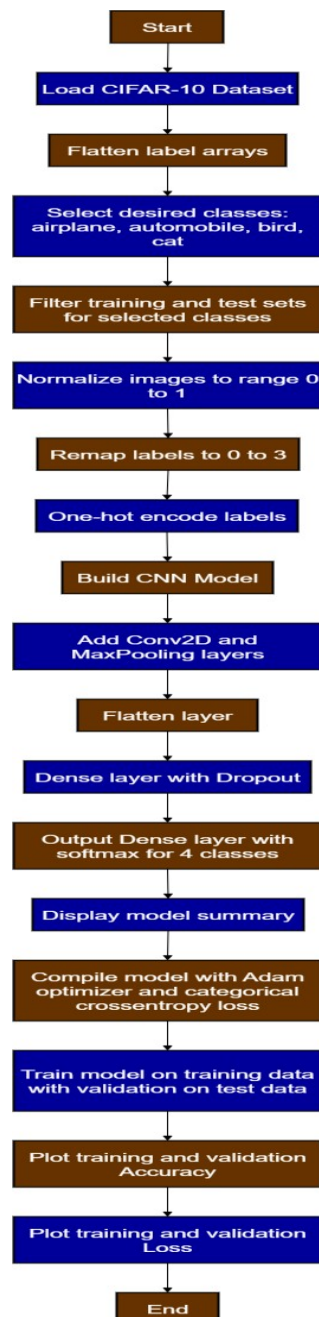
**Constituent of Symbiosis International (Deemed University),
Pune**

Flowchart



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Part-1



**Constituent of Symbiosis International (Deemed University),
Pune**



Name-Janay Rawal
PRN-22070123049
ENTC-A (G1)

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Program with Output:

Program:

```
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from keras.datasets import cifar10
from keras.utils import to_categorical
import numpy as np
import matplotlib.pyplot as plt

# -----
# STEP 1: Load and Preprocess Data
# -----
(X_train, y_train), (X_test, y_test) = cifar10.load_data()

# Flatten label arrays
y_train = y_train.flatten()
y_test = y_test.flatten()

# Select desired classes (0=airplane, 1=automobile, 2=bird, 3=cat)
selected_classes = [0, 1, 2, 3]
train_filter = np.isin(y_train, selected_classes)
test_filter = np.isin(y_test, selected_classes)

# Apply filters
X_train, y_train = X_train[train_filter], y_train[train_filter]
X_test, y_test = X_test[test_filter], y_test[test_filter]

# Normalize images
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0
```

**Constituent of Symbiosis International (Deemed University),
Pune**



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

```
# Remap labels to 0-3
class_map = {c: i for i, c in enumerate(selected_classes)}
y_train = np.vectorize(class_map.get)(y_train)
y_test = np.vectorize(class_map.get)(y_test)

# One-hot encode
y_train = to_categorical(y_train, num_classes=4)
y_test = to_categorical(y_test, num_classes=4)

# -----
# STEP 2: Build CNN Model
# -----
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(64, activation='relu'),
    Dropout(0.5),
    Dense(4, activation='softmax')
])

# Display model summary
model.summary()

# -----
# STEP 3: Compile and Train
# -----
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University),
Pune

```
# Display model summary
model.summary()

# -----
# STEP 3: Compile and Train
# -----
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

history = model.fit(
    X_train, y_train,
    epochs=10,          # increased for better learning
    batch_size=64,
    validation_data=(X_test, y_test)
)

# -----
# STEP 4: Plot Accuracy and Loss
# -----
plt.figure(figsize=(12, 5))

# Accuracy plot
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# Accuracy plot
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# Loss plot
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.show()
```

Output:



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Model: "sequential_1"

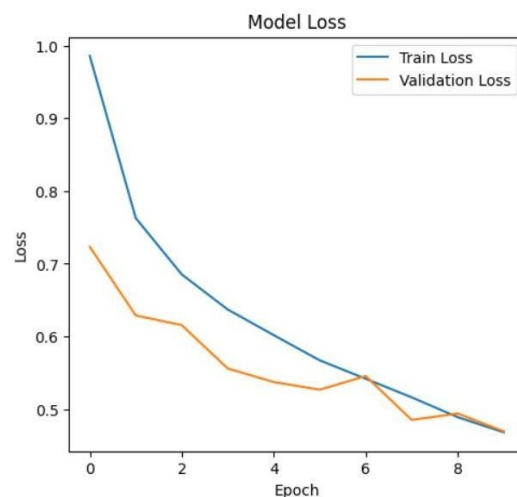
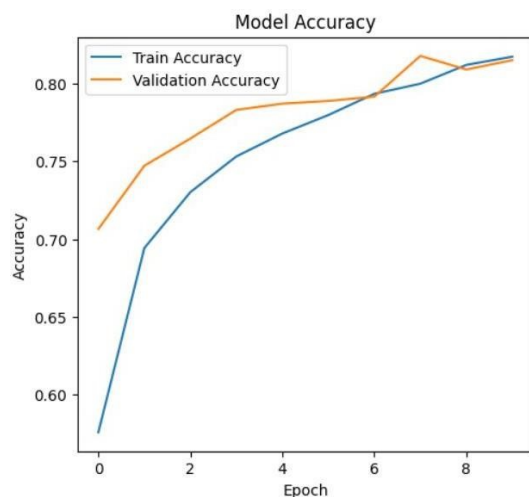
Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d_2 (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_3 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 6, 6, 64)	0
flatten_1 (Flatten)	(None, 2304)	0
dense_2 (Dense)	(None, 64)	147,520
dropout_1 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 4)	260

Total params: 167,172 (653.02 KB)
Trainable params: 167,172 (653.02 KB)
Non-trainable params: 0 (0.00 B)

Epoch 1/10
313/313 — 33s 96ms/step - accuracy: 0.4809 - loss: 1.1300 - val_accuracy: 0.7065 - val_loss: 0.7234
Epoch 2/10
313/313 — 36s 80ms/step - accuracy: 0.6851 - loss: 0.7833 - val_accuracy: 0.7473 - val_loss: 0.6289
Epoch 3/10
313/313 — 42s 82ms/step - accuracy: 0.7251 - loss: 0.6965 - val_accuracy: 0.7648 - val_loss: 0.6155
Epoch 4/10
313/313 — 26s 83ms/step - accuracy: 0.7514 - loss: 0.6366 - val_accuracy: 0.7832 - val_loss: 0.5559

Constituent of Symbiosis International (Deemed University), Pune

Epoch 5/10
313/313 — 25s 81ms/step - accuracy: 0.7674 - loss: 0.6016 - val_accuracy: 0.7872 - val_loss: 0.5372
Epoch 6/10
313/313 — 41s 80ms/step - accuracy: 0.7797 - loss: 0.5667 - val_accuracy: 0.7890 - val_loss: 0.5267
Epoch 7/10
313/313 — 25s 80ms/step - accuracy: 0.7939 - loss: 0.5447 - val_accuracy: 0.7918 - val_loss: 0.5455
Epoch 8/10
313/313 — 25s 79ms/step - accuracy: 0.7971 - loss: 0.5266 - val_accuracy: 0.8180 - val_loss: 0.4851
Epoch 9/10
313/313 — 26s 83ms/step - accuracy: 0.8110 - loss: 0.4956 - val_accuracy: 0.8092 - val_loss: 0.4939
Epoch 10/10
313/313 — 25s 80ms/step - accuracy: 0.8159 - loss: 0.4679 - val_accuracy: 0.8152 - val_loss: 0.4696



Conclusion/Inferences:



Name-Janay Rawal
PRN-22070123049
ENTC-A (G1)

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

A custom Convolutional Neural Network (CNN) is an effective and flexible approach for image classification. By combining convolutional layers for feature extraction, pooling layers for dimensionality reduction, and fully connected layers for decision-making, CNNs can automatically learn both low-level and high-level patterns from images. Designing a custom architecture allows tailoring the network to specific datasets, balancing accuracy and computational efficiency, and incorporating regularization techniques to prevent overfitting. Overall, CNNs provide a powerful framework for solving complex image recognition tasks, making them a cornerstone of modern computer vision applications.